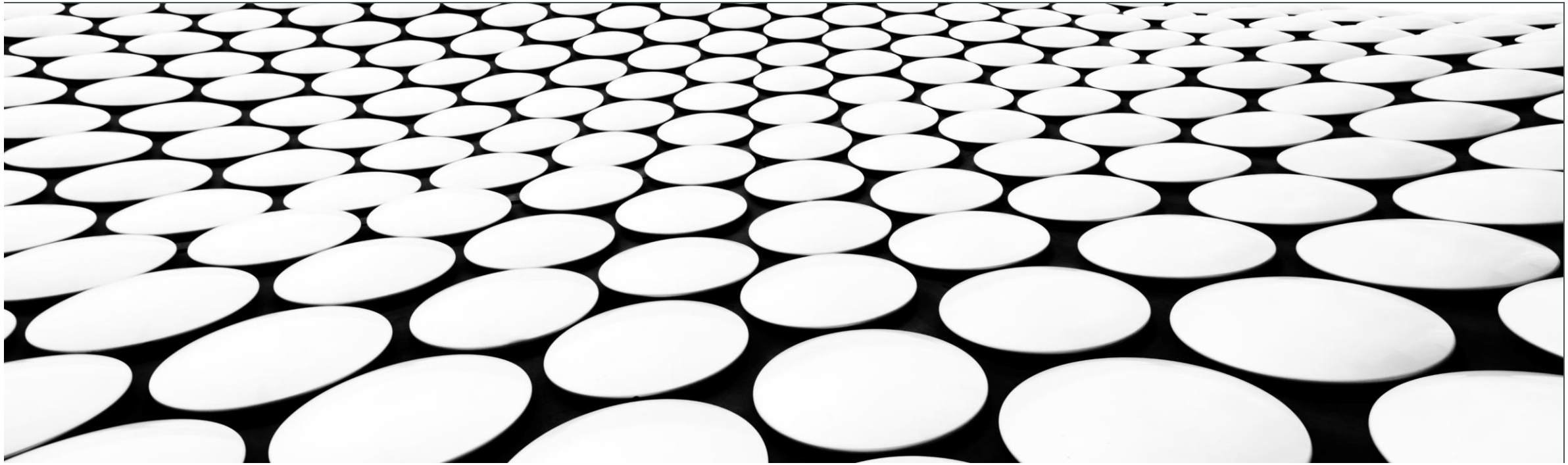
자료구조와 알고리즘

확장형 자료구조 - 스택/큐/그래프/트리/그래프 순회



학습 개요

그래프, 트리

1. 스택
2. 큐
3. 그래프
4. 트리
5. 그래프 순회

- 스택 구조란?
  - 아이스크림 콘에 여러 가지 맛을 쌓을 때, 가장 먼저 넣은 맛을 가장 나중에 먹을 수 있는 구조

초콜릿을 먹으려면 순서대로 쌓여 있는 딸기 → 바닐라 → 커피를 차례대로 먹어야 초콜릿을 먹을 수 있다.

- 스택의 개념
  - 스택(Stack) 자료구조는 한쪽 끝이 막힌 형태(ex : 한쪽 끝이 막힌 주차장, 종이컵 수거함 등)
  - 입구가 하나이기 때문에 먼저 들어간 것이 가장 나중에 나오는 구조(선입후출, 후입선출)

그림 6-1 스택의 실생활 사례

- 스택의 개념
  - 종이컵 수거함에 종이컵을 넣고 빼는 예

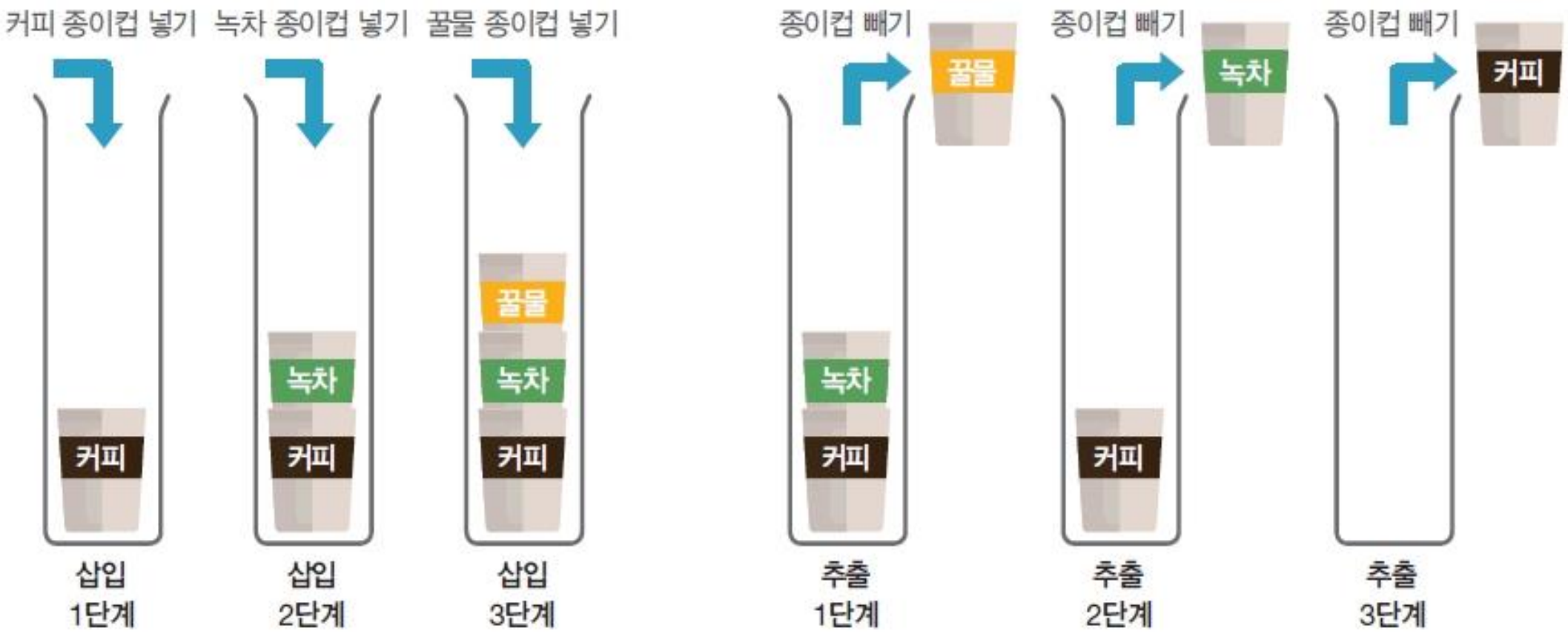


그림 6-2 종이컵 수거함의 작동 원리

- 스택 원리
  - 스택 기본 구조
    - 스택에 데이터를 삽입하는 작동 : push
    - 스택에 데이터를 추출하는 작동 : pop
    - 스택에 들어 있는 가장 위의 데이터 : top

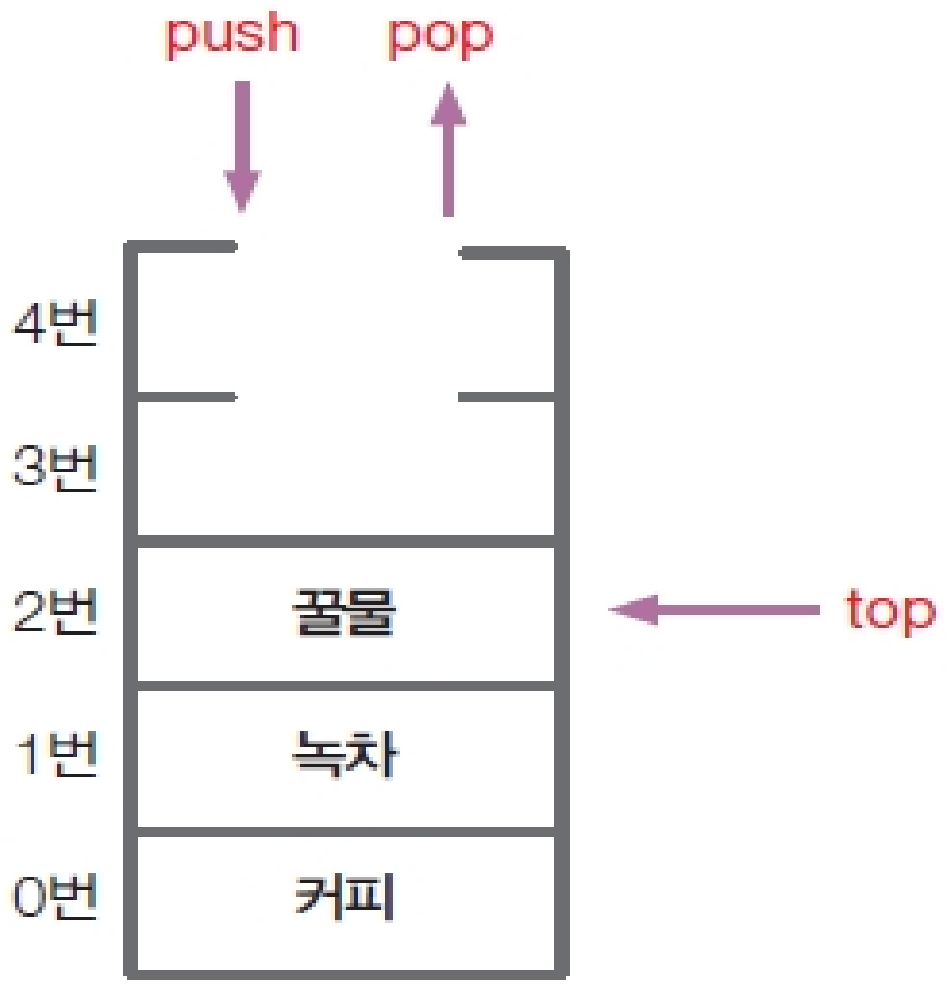


그림 6-3 스택 기본 구조

- 데이터 삽입 : push

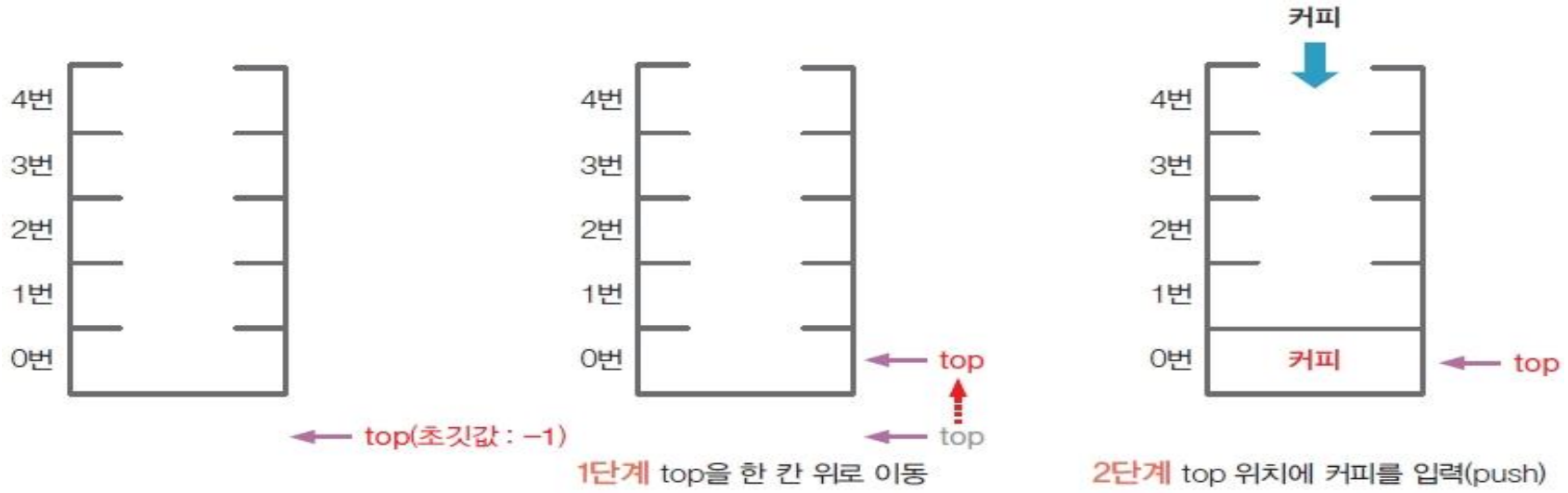


그림 6-4 스택에 데이터를 삽입(push)하는 과정

- 데이터 추출 : pop

그림 6-5 스택에서 데이터를 추출(pop)하는 과정

- 스택 생성
  - 배열 크기를 지정한 후 빈 스택 생성

```
stack = [None, None, None, None, None]
top = -1
```

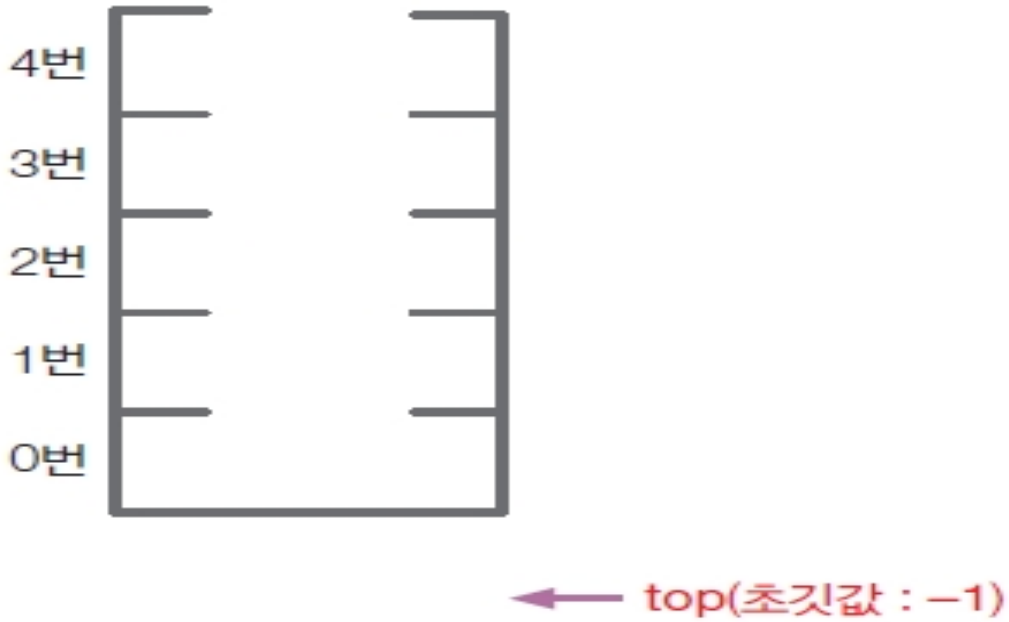
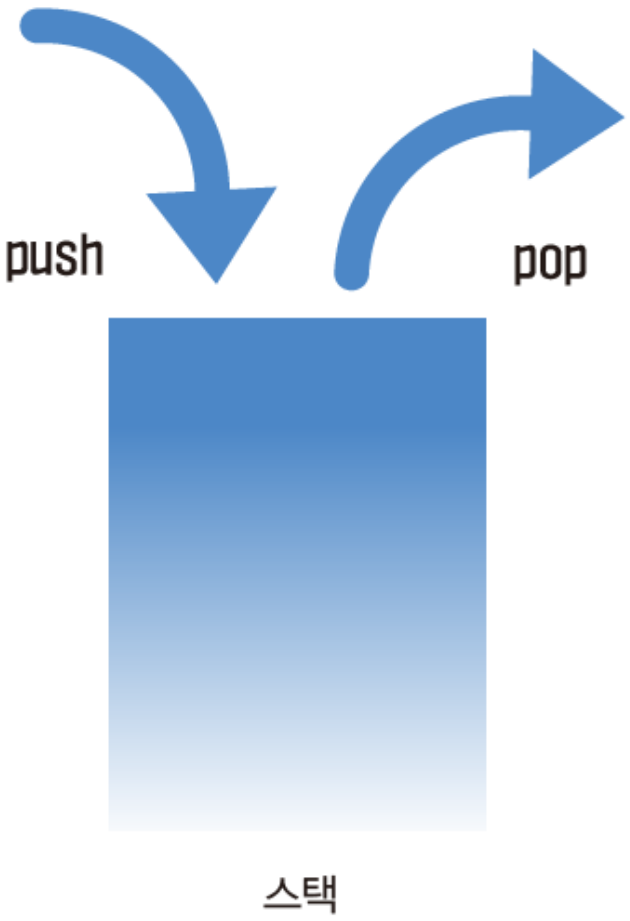


그림 6–6 크기가 5칸인 스택의 초기 상태

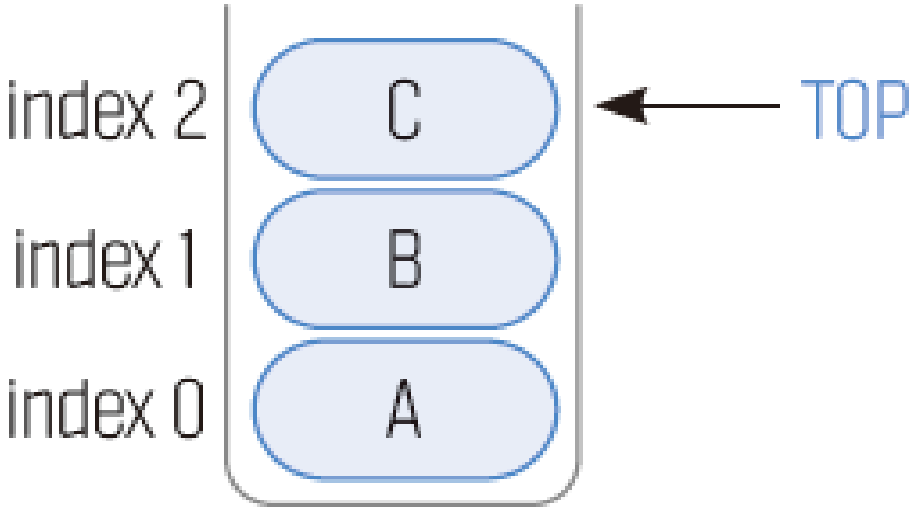
스택

- 기본 자료구조에 규칙을 포함한 자료구조
- 규칙을 가지고 데이터를 저장
- 규칙
  - 가장 마지막에 데이터 저장(PUSH)
  - 가장 마지막 데이터를 읽음(POP)
  - Last In First Out(LIFO)

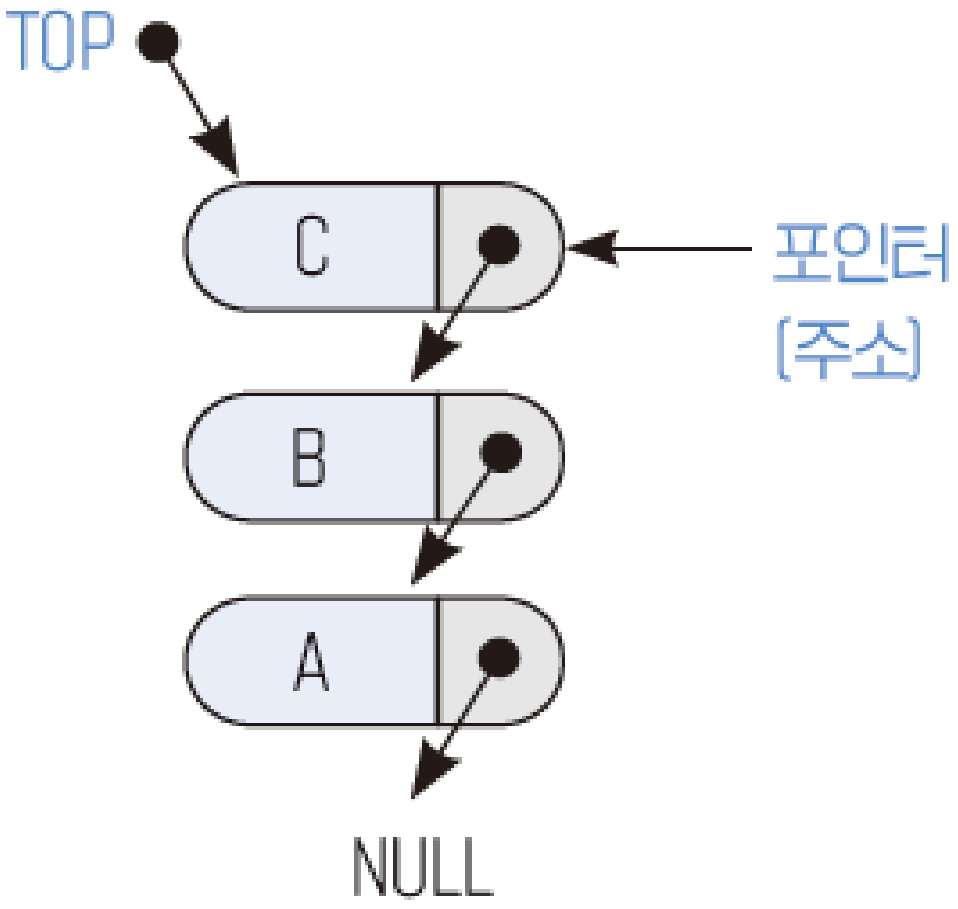


- 스택(Stack)의 구현
  - 배열 자료구조를 이용한 스택 구현
    - 스택의 크기를 미리 정해놓고 사용
  - 연결 리스트 자료구조를 이용한 스택 구현
    - 스택의 크기가 한정되지 않고 삽입 삭제가 용이
  - 각 프로그래밍 언어에서 제공하는 자료구조(자료형)를 이용하여 구현

스택(Stack)의 구현



배열을 이용한 스택



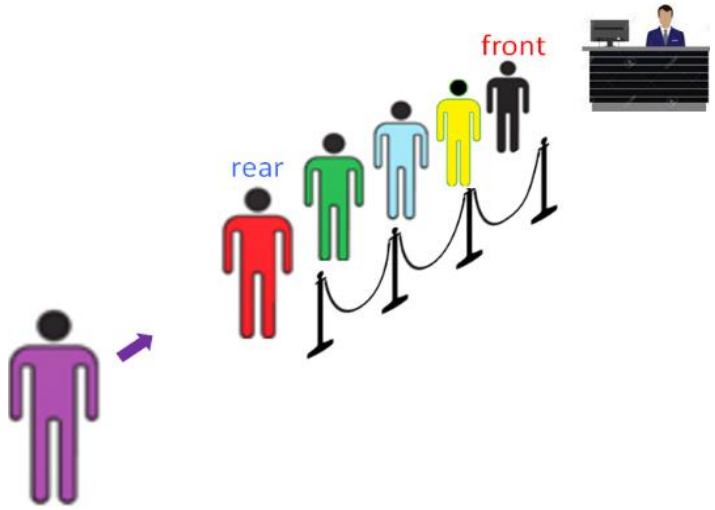
연결 리스트를 이용한 스택

스택

- 스택을 이용한 문제 해결
- 괄호 확인
  - print("계산결과는{0}입니다.".format((int(a)+int(b))))
    - 왼쪽 괄호를 만나면 스택에 PUSH()한다.
    - 오른쪽 괄호를 만나면 스택에서 POP()한다.
    - 스택에 데이터가 없으면, 괄호의 개수가 맞는 것이다.
- 우선순위 연산(후위 표기법 사용)
  - A*B-C/D 연산 → ((AB)*(CD)/)- → AB*CD/-
    - 피 연산자는 PUSH()
    - 연산자를 만나면 피연산자를 POP()하여 연산 후 다시 PUSH()
    - 마지막 POP()으로 연산 결과 출력
- 역순 출력
  - abcdefg → gfedcba
    - 문자가 끝날 때까지 PUSH()
    - 스택에 데이터가 없을 때까지 POP()

3.3 큐

- 큐(Queue): 삽입과 삭제가 양 끝에서 각각 수행되는 자료구조
- 일상생활의 관공서, 은행, 우체국, 병원 등에서 번호표를 이용한 줄 서기
- 선입 선출(First-In First-Out, FIFO)



- **큐란?**
  - 기차가 터널에 들어가는 순서대로 터널을 빠져나오고, ATM기에서 줄을 선 순서대로 예금을 인출하는 것처럼 큐는 먼저 들어간 것이 먼저 나오는 구조를 의미

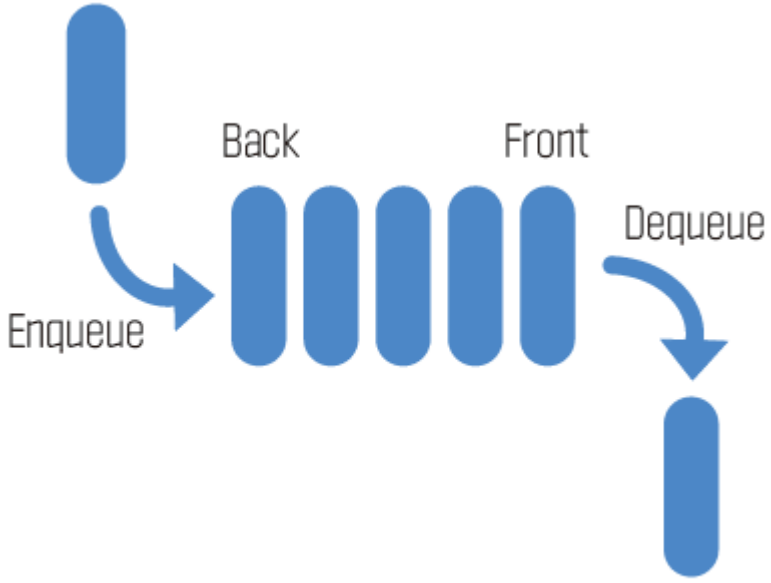
- **큐의 개념**
  - 큐(Queue) 자료구조는 입구와 출구가 따로 있는 원통 형태
    - **스택과 큐의 차이**

그림 7-1 스택과 큐 차이점

그림 7-1 큐 개념의 일상 예

큐(QUEUE)

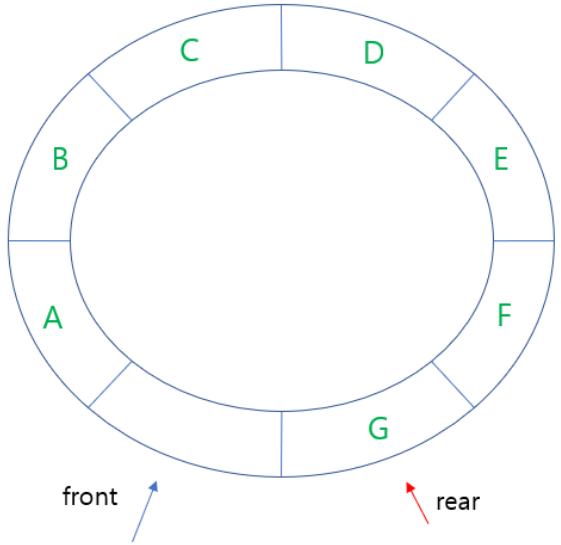
- 기본 자료구조에 규칙을 포함한 자료구조
- 규칙을 가지고 데이터 저장
- 규칙
  - 가장 마지막에 데이터 저장
  - 가장 처음 데이터를 읽음
  - First In First Out(FIFO)



큐

- 큐(Queue)의 구현
  - 배열 자료구조를 이용한 큐 구현
    - 큐의 크기를 미리 정해놓고 사용
    - 선형 큐 → 삭제 후 데이터의 위치 이동 또는 재사용 불가 → 순환 큐 구현

	A	B	C		
Index	0	1	2	3	4
	↑ front		↑ rear		



  - 연결 리스트 자료구조를 이용한 큐 구현
    - 큐의 크기가 한정되지 않음
  - 각 프로그래밍 언어에서 제공하는 자료구조(자료형)를 이용하여 구현

큐

- 큐의 사용
  - 우선순위가 같은 작업 예약
  - 은행업무
  - 콜센터 고객 대기시간
  - 프로세스 관리

```python
w_no=100
queue=[]

def enqueue(data):
    queue.append(data)

def dequeue():
    dequeue_object=None
    if len(queue)==0:
        print("Queue is Empty")

    else:
        dequeue_object=queue[0]
        queue.remove(queue[0])

        return dequeue_object

while True:
    a=int(input("대기표 뽑기:1,업무처리:2,종료:3\n"))

    if a==1:
        enqueue(w_no)
        w_no+=1
        print("현재 대기표번호:",w_no)
    elif a==2:
        b=dequeue()
        print("현재 업무처리번호:",b)
    else:
        break
```

파이썬 코드

```
대기표 뽑기:1, 업무처리:2, 종료:3
1
현재 대기표번호: 101
대기표 뽑기:1, 업무처리:2, 종료:3
1
현재 대기표번호: 102
대기표 뽑기:1, 업무처리:2, 종료:3
1
현재 대기표번호: 103
대기표 뽑기:1, 업무처리:2, 종료:3
2
현재 업무처리번호: 100
대기표 뽑기:1, 업무처리:2, 종료:3
2
현재 업무처리번호: 101
대기표 뽑기:1, 업무처리:2, 종료:3
1
현재 대기표번호: 104
대기표 뽑기:1, 업무처리:2, 종료:3
2
현재 업무처리번호: 102
대기표 뽑기:1, 업무처리:2, 종료:3
3
```

결과 화면

그래프

- 그래프(Graph) 이론
  - 그래프 이론의 최초의 연구 : 쾨니히스베르크의 다리 문제
  - 쾨니히스베르크의 다리 문제

    “7개의 다리는 한 번씩만 건너서 쾨니히스테르크의 모든 지역을 돌아 제자리로 돌아올 수 있을까?”
  - 1725년 레온하르트 오일러의 증명

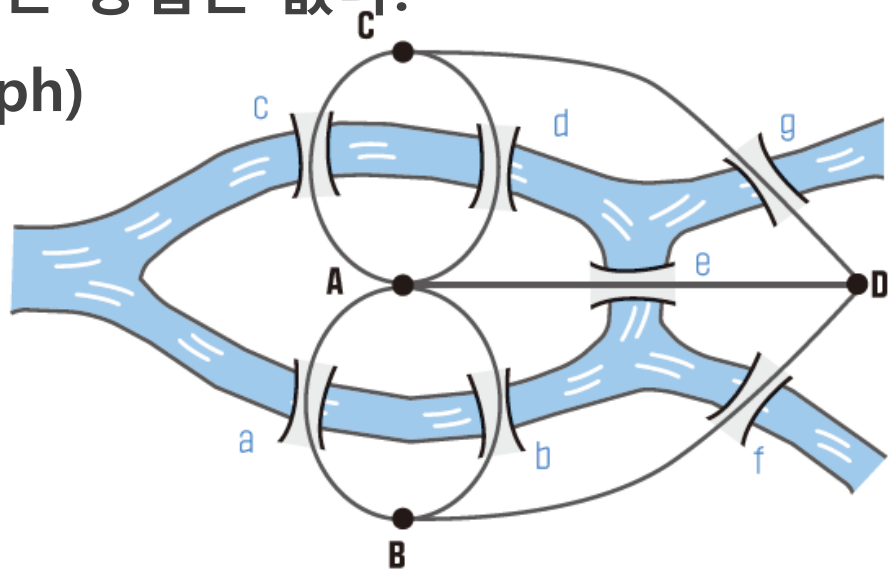
    “모든 다리를 한 번만 거치고 첫 시작 위치로 돌아오는 길은 없다”
    - 이 문제의 해답을 찾기 위한 연구의 부산물로 그래프 이론이라는 수학의 한 분야가 생겨남
    - 논문에 포함된 이 스케치는 현대에 이르러 그래프 구조의 원형이 됨
    - 그래프는 수백년 동안 수학, 과학, 공학, 사회, 경제학에 이르기까지 폭넓은 분야에서 응용됨
    - 컴퓨터 분야에서도 그래프 이론을 토대로 한 알고리즘이 만들어짐

오일러의 증명

- 그림을 단순화 하고 육지를 정점(Vertex)으로 하며, 각 육지를 잇는 다리를 간선(Edge)으로 표현(4개의 정점과 7개의 간선)
- 한붓 그리기 규칙 : 연결되어 있는 선이 홀수 개인 점이 없거나 두 개만 있는 도형에서만 가능
- 쾨니히스테르크 문제에서 각 정점 4개 모두 간선이 홀수 개이므로 한붓 그리기 불가능

  → 모든 다리를 한 번만 거치고 첫 시작 위치로 돌아오는 방법은 없다.
- 모든 간선을 한 번씩 방문하는 유한 그래프(Finite Graph)

  : 오일러 경로(Eulerian Trail/Eulerian Path)

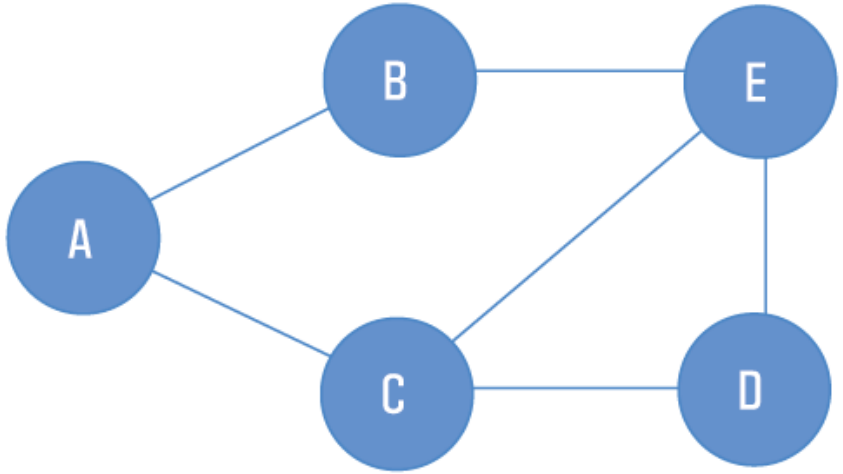


쾨니히스베르크 다리 건너기 문제

그래프

- 그래프의 정의
  - 그래프는 정점과 간선 모음들의 결합
  - 정점의 집합을 V, 간선의 집합을 E, 그래프를 G라고 했을 때 G=(V,E)
- 그래프 용어
  - 인접(Adjacent) : 간선으로 연결되어 있는 두 정점 , 이웃 관계에 있다고 말할 수 있다.

    (A,B), (A,C), (B,E), (C,E), (C,D), (D,E)
  - 경로(Path) : 간선을 통해 서로 이웃된 각 정점들로 그래프 안에서 형성된 길을 말한다. 정점 A에서 E까지는 A, B, E와
    A, C, E 두 개의 경로를 이루고 있다.



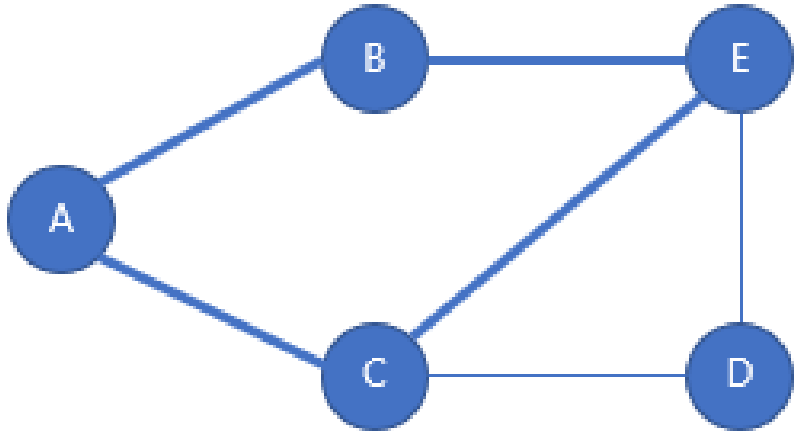
그래프 용어

- 차수 (degree) : 정점에 속한 간선들의 수를 말한다.

  정점 A,B,D의 차수는 2 이고, 정점 E,C 의 차수는 3

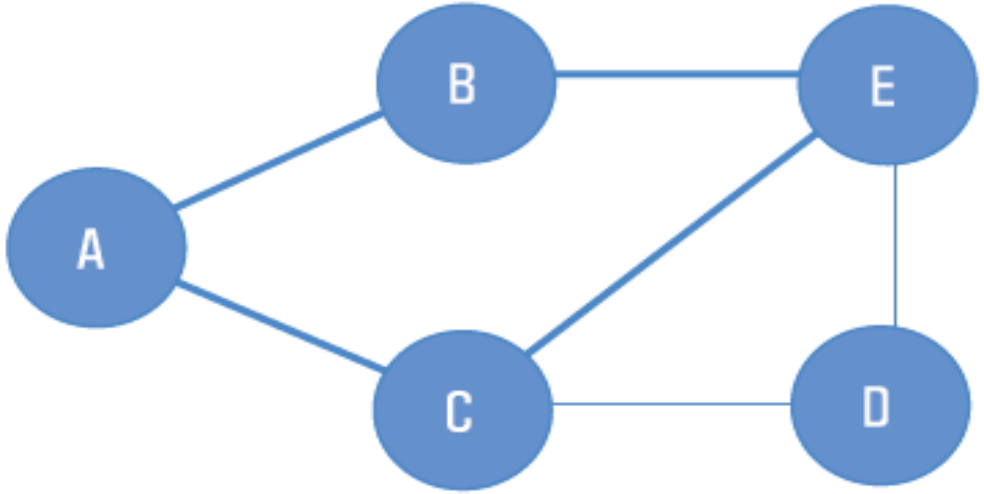
- 연결 (Connected) : 무 방향 그래프 내에서 두 정점 사이의 경로가 존재하면 두 정점이 연결 되었다고 말한다. 그래프 내의 모든 정점들이 모두 연결 되어 있으면 그래프가 연결되었다고 말한다.

- 사이클 (Cycle) : 어느 경로가 정점 하나를 두 번 이상 거치도록 되어 있는 경로, 처음과 마지막 정점이 같은 단순 경로를 말한다.

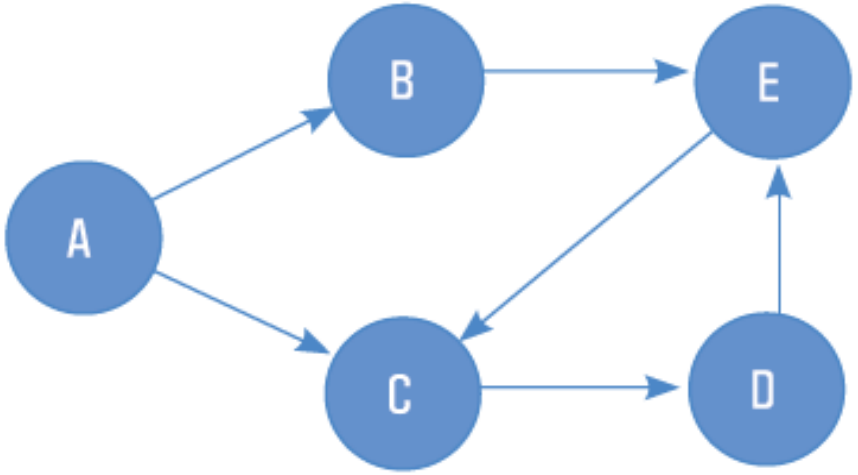


그래프 종류

- 무방향 그래프(Undirected Graph) : 간선에 방향성이 없는 그래프, 간선을 나타내는 정점의 쌍에 순서가 없다. (A,B)와 (B,A)는 동일한 간선을 표현
- 방향 그래프(Directed Graph) : 간선에 방향성이 있는 그래프, 간선을 나타내는 정점의 쌍에 순서가 있다. <A,B>로 표현하며, 이때 A는 꼬리(Tail), B는 머리(Head)로 표현하기도 한다.



무방향 그래프



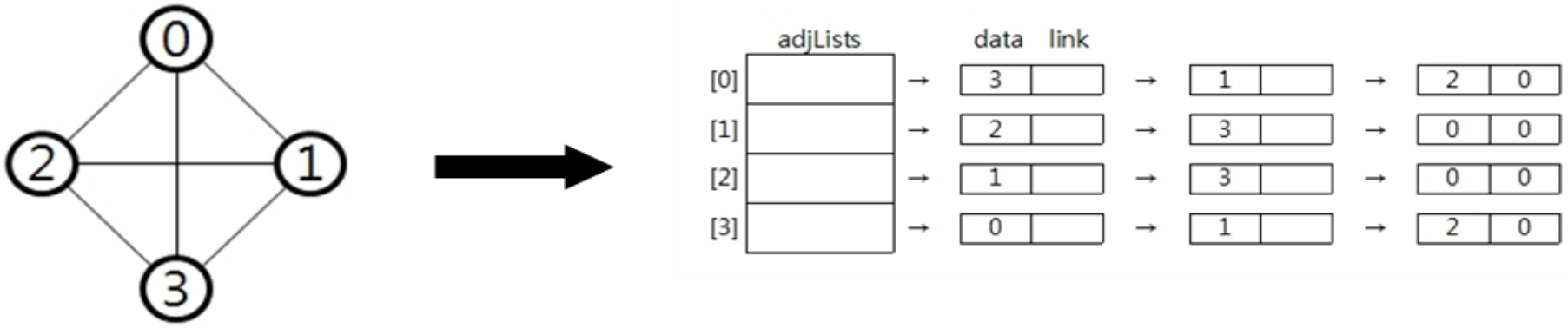
방향 그래프

그래프 (GRAPH)

- 그래프 자료구조
  - 그래프 특징과 규칙을 갖도록 데이터를 저장 → 자료구조
  - 자료형과 기본 자료구조를 이용하여 데이터 저장
  - 프로그래밍 언어에서 제공하는 자료형을 이용하여 표현

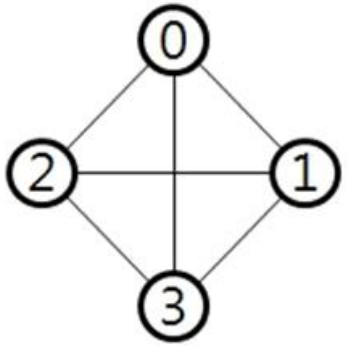
그래프 표현 - 인접 리스트

- 배열과 연결 리스트 자료구조 사용
- 적은 메모리 공간을 요구하지만, 인접 행렬보다 접근이 느리다.
- 연결 리스트 자료구조를 통하여 그래프를 표현한다.
- 그래프의 각 정점마다 헤드 노드를 두어 각 정점을 할당한다.
- 각 정점과의 사이에 정점이 존재하는 정점들을 헤드 노드 뒤에 연결한다.
- 무방향 그래프일 경우 헤드 노드 뒤에 연결된 노드의 개수가 그 노드의 차수이다.

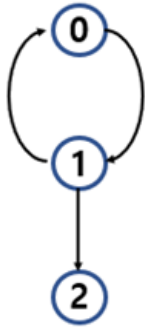


그래프 표현 – 배열

- 2차원 배열 자료구조 사용
- 많은 메모리 공간을 요구하지만, 인접 리스트 보다 접근이 빠르다.
- 정점 수만큼의 행과 열을 가진 행렬을 이용하여 표현한다.
- 정점 i 와 j 사이에 간선이 존재하면 i 번째 행과 j 번째 열의 원소가 1로 표현한다.
- 간선이 존재하지 않으면 I 번째 행과 j 번째 열의 원소가 0 으로 표현한다.
- n개의 정점을 가진 무방향 그래프의 인접 행렬에서 진입 차수는 행의 합이다.



$$\begin{array}{c|c} & 0\,1\,2\,3 \\ \hline 0 & 0\,1\,1\,1 \\ 1 & 1\,0\,1\,1 \\ 2 & 1\,1\,0\,1 \\ 3 & 1\,1\,1\,0 \end{array}$$



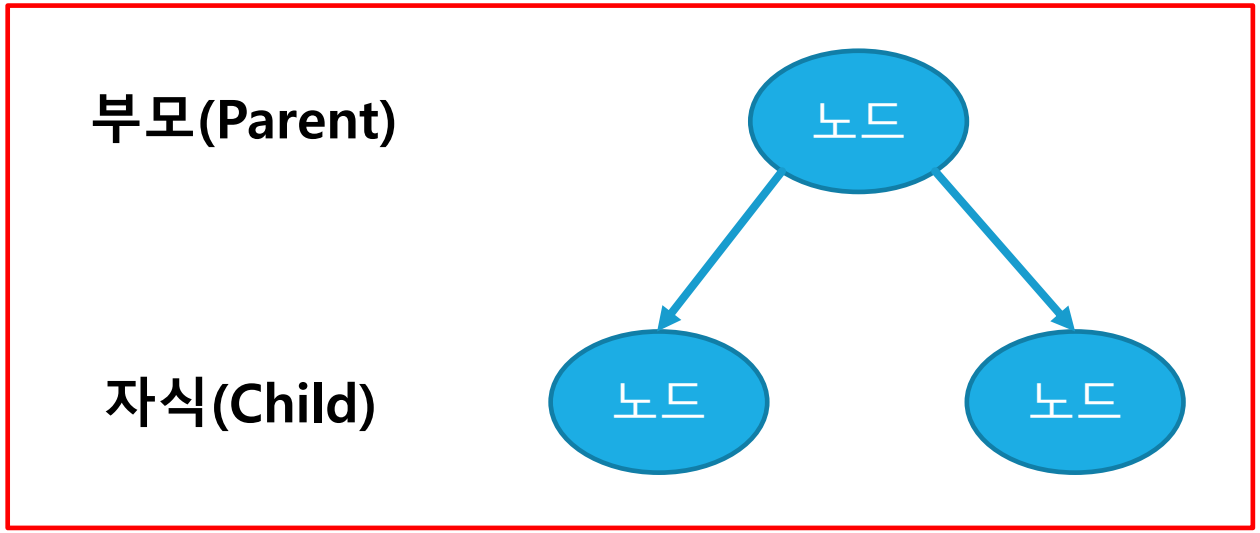
$$\begin{array}{c|c} & 0\;\;1\;\;2 \\ \hline 0 & 0\;1\;0 \\ 1 & 1\;0\;1 \\ 2 & 0\;0\;0 \end{array}$$

그래프

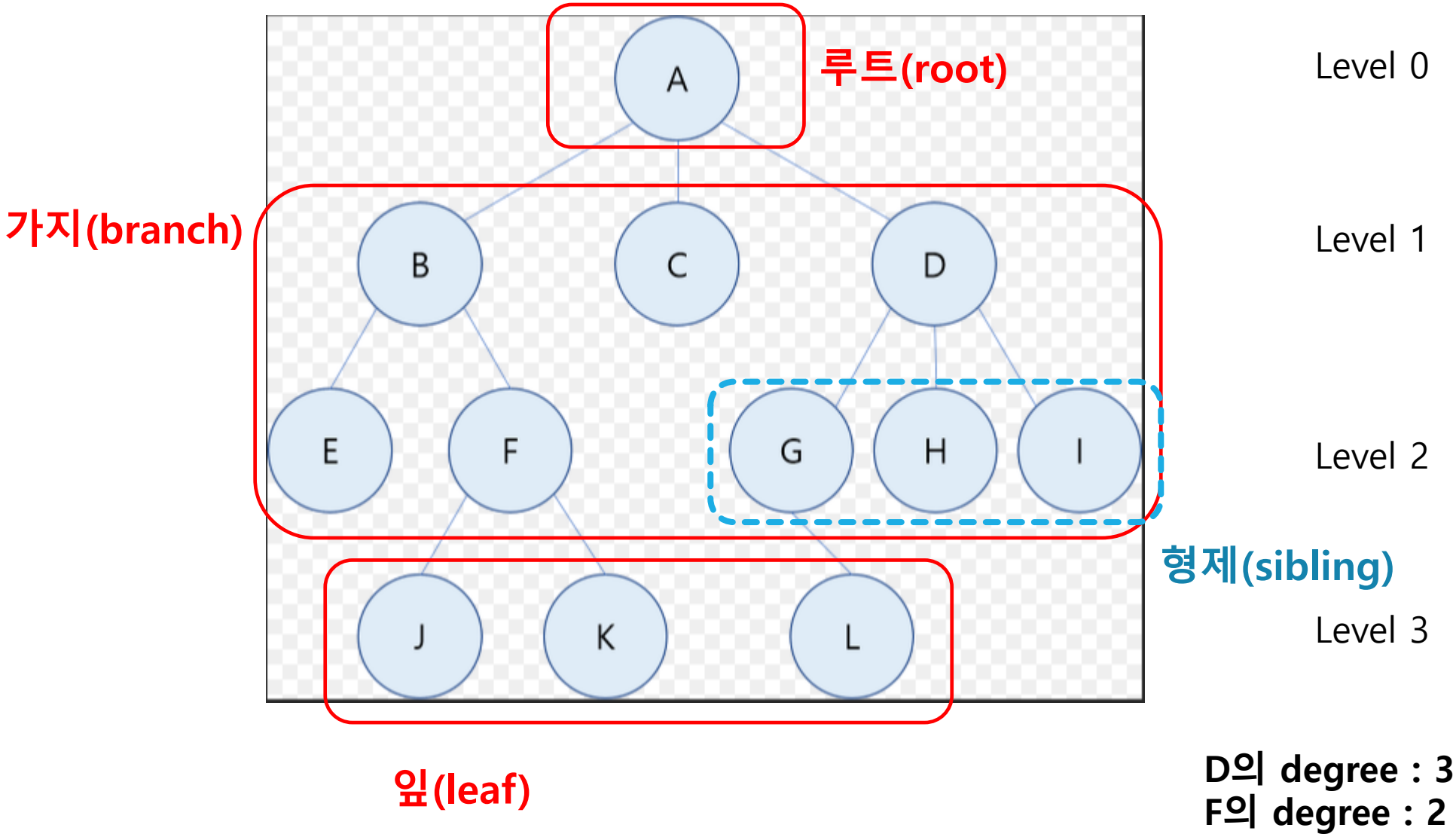
- 그래프 자료구조를 이용한 문제 해결 알고리즘
  - 최단 거리 문제 – 프림 알고리즘, 크러스컬 알고리즘, 다익스트라 알고리즘
  - 탐색 문제 – D F S 탐색 알고리즘, BFS 탐색 알고리즘

트리

- 트리(Tree)
  - 노드와 간선으로 구성
  - 그래프의 일종으로 여러 노드가 한 노드를 가리킬 수 없는 구조
- 트리의 종류
  - B 트리
    - 자식 노드가 2개 이상
  - 이진트리
    - 자식 노드가 2개 이하(0, 1, 2)
  - 완전 이진 트리
    - 자식 노드를 추가할 때 왼쪽부터 오른쪽으로 추가
  - 균형 이진 트리
    - 자식 노드의 추가 위치는 상관없지만, 왼쪽과 오른쪽의 레벨 차이가 1 이하



트리

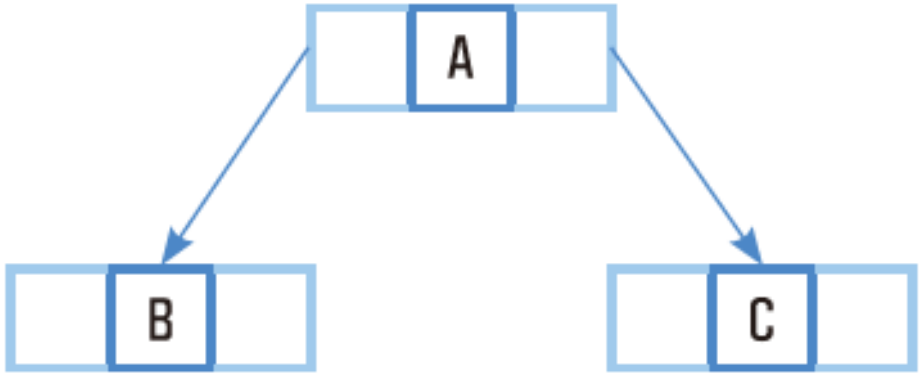


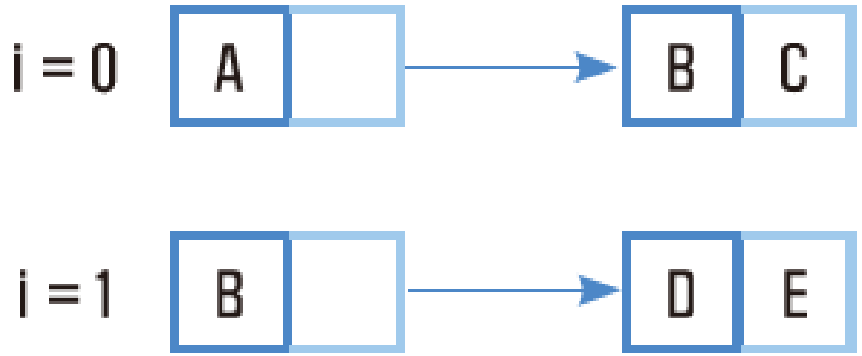
D의 degree : 3
F의 degree : 2

트리

- 트리 자료구조
  - 트리 특징과 규칙을 갖도록 데이터를 저장 → 자료구조
  - 자료형과 기본 자료구조를 이용하여 데이터 저장
  - 프로그래밍 언어에서 제공하는 자료형을 이용하여 표현

- 트리 표현
  - 인접 리스트
    - 왼쪽 자식을 가리키는 포인터, 오른쪽 자식을 가리키는 포인터
  - 배열
    - 1차원 배열
      - 자신의 부모 노드만 저장하는 방법
    - 2차원 배열
      - A[i][0] : 왼쪽 자식 노드,
        A[i][1] : 오른쪽 자식 노드





트리

- 트리 자료구조를 이용한 문제 해결 알고리즘
  - 효율적인 데이터 삽입, 삭제
  - 탐색 문제 – 이진 탐색 알고리즘
  - 정렬 문제 – 힙 정렬 알고리즘
  - 압축 문제 – 허프만 압축 알고리즘

	그래프	트리
정의	노드와 그 노드를 연결하는 간선을 하나로 모아 놓은 자료구조	그래프의 한 종류
방향성	방향/무방향 그래프 모두 존재	방향 그래프
사이클	사이클 가능 순환/비순환 그래프 모두 가능	사이클 불가능 비순환 그래프
루트 노드	루트 노드 개념이 없음	한 개의 루트 노드만 존재
부모-자식	부모-자식 개념 없음	부모-자식 관계
모델	네트워크 모델	계층 모델
간선의 수	그래프에 따라 간선의 수가 다름 간선이 없을 수도 있음	노드가 n인 트리는 항상 n-1의 간선을 가짐

그래프 순회

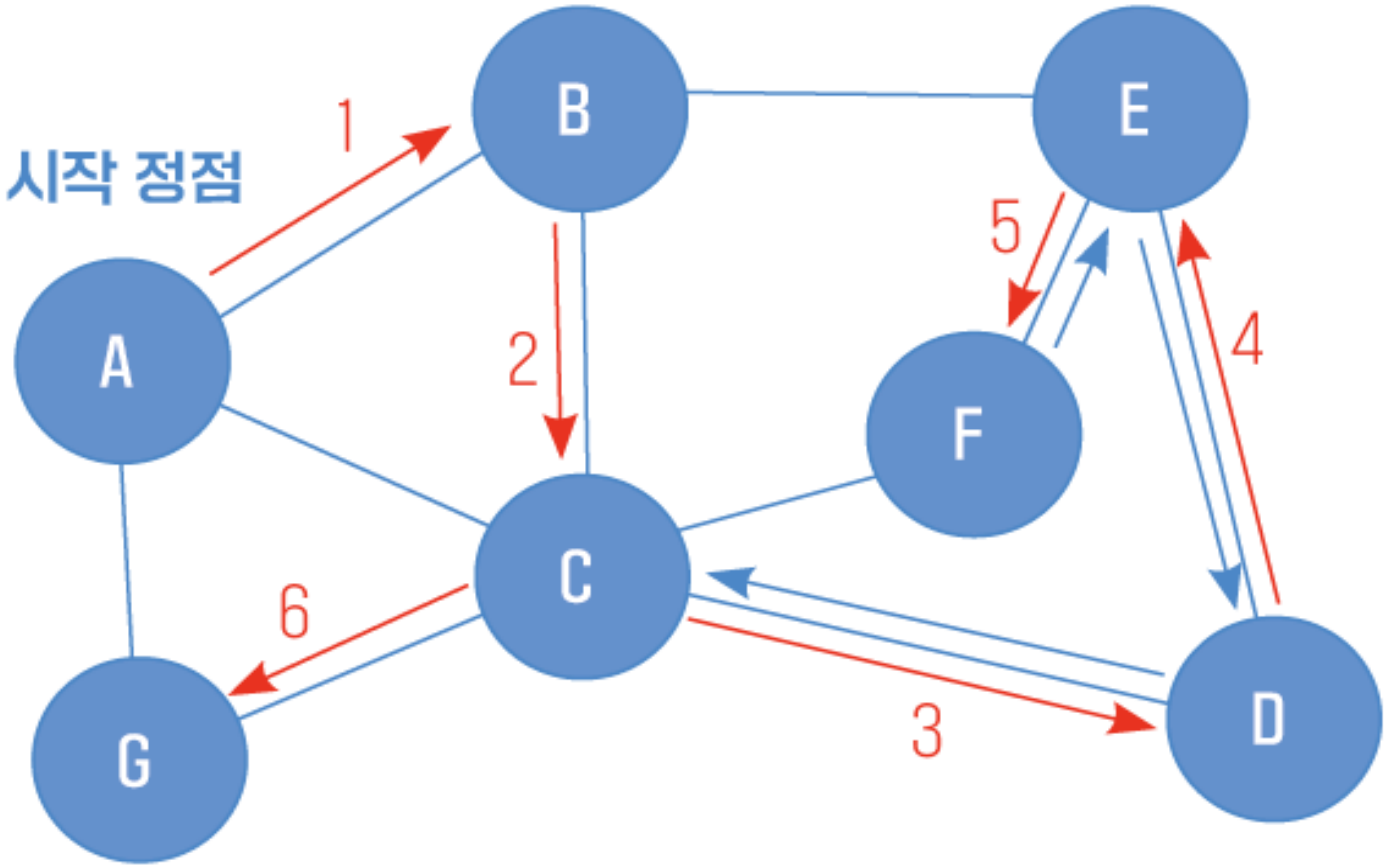
- 그래프 또는 트리와 같은 연결 구조에서 모든 정점(노드)을 순차적으로 탐색
- 목적 : 특정 노드를 탐색
- 깊이 우선 탐색(DFS : Depth First Search)
- 너비 우선 탐색(BFS : Breadth First Search)
- 경험적 탐색

- 깊이 우선 순회(DF S : Depth First Search)
  - 시작 노드를 정하고 방문하지 않은 노드가 존재할 때까지 방문하는 방법

- 현재 노드의 인접 노드 중 방문하지 않은 노드를 방문한다.
- 방문한 노드의 인접 노드 중 더 이상 방문하지 않은 인접 노드가 존재하지 않으면 방문했던 경로로 되돌아간다.
- 방문하지 않은 노드가 더 이상 존재하지 않으면 종료한다.

그래프 순회

- 깊이 우선 순회
  - 시작 노드에서 한 방향으로 마지막 노드까지 방문하고,
    방문 경로를 되돌아가면서 방문하지 않은 노드를 확인한 후 방문한다.

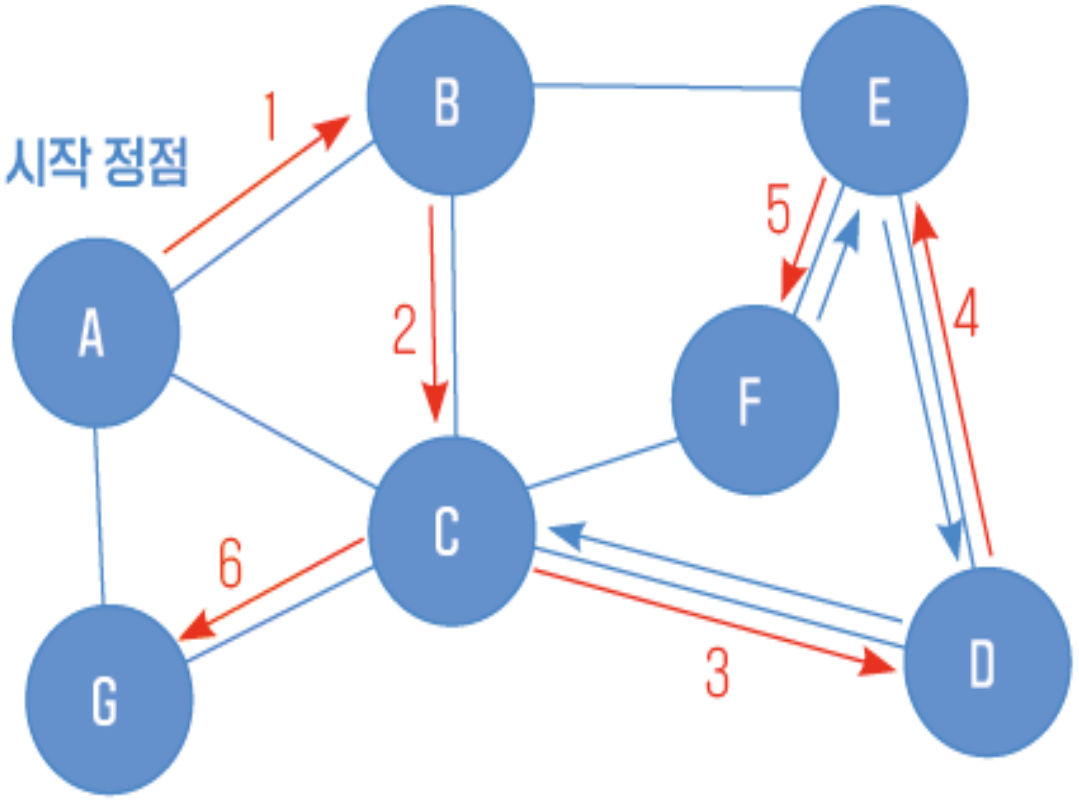


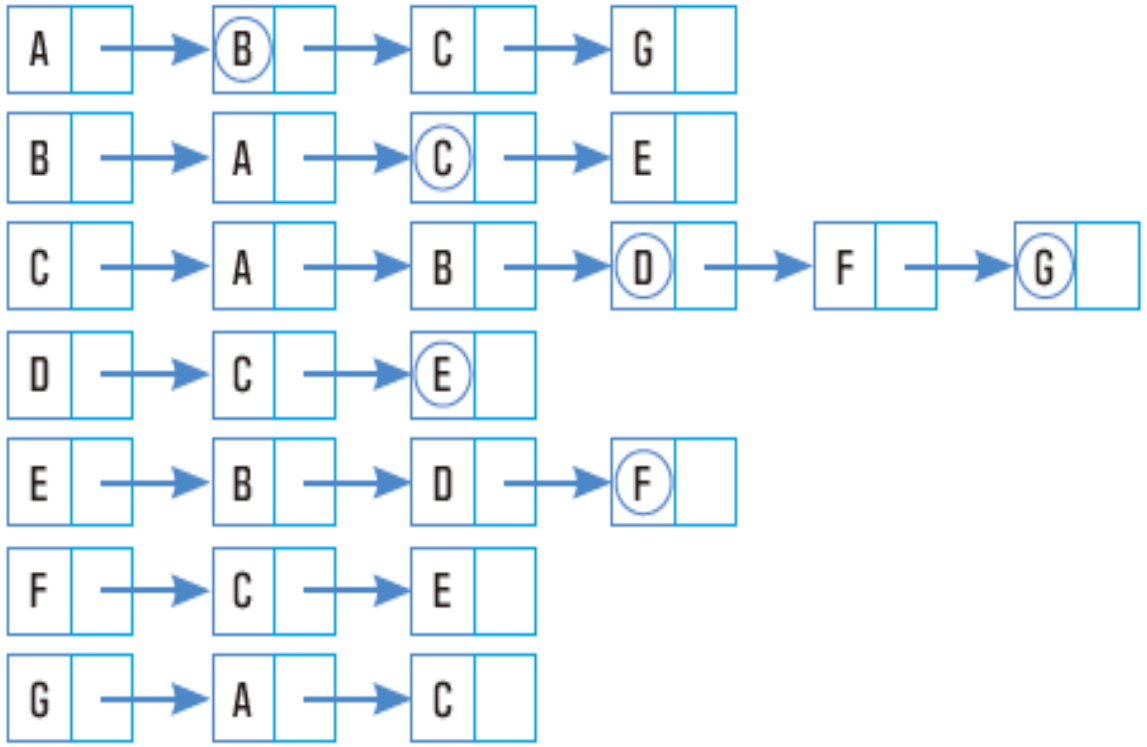
순회 순서

A → B → C → D → E → F
→ E → D → C → G → C → B → A

종료 조건
시작 정점으로 돌아옴

▪ 기본 자료구조로 표현한 깊이 우선 순회 수행

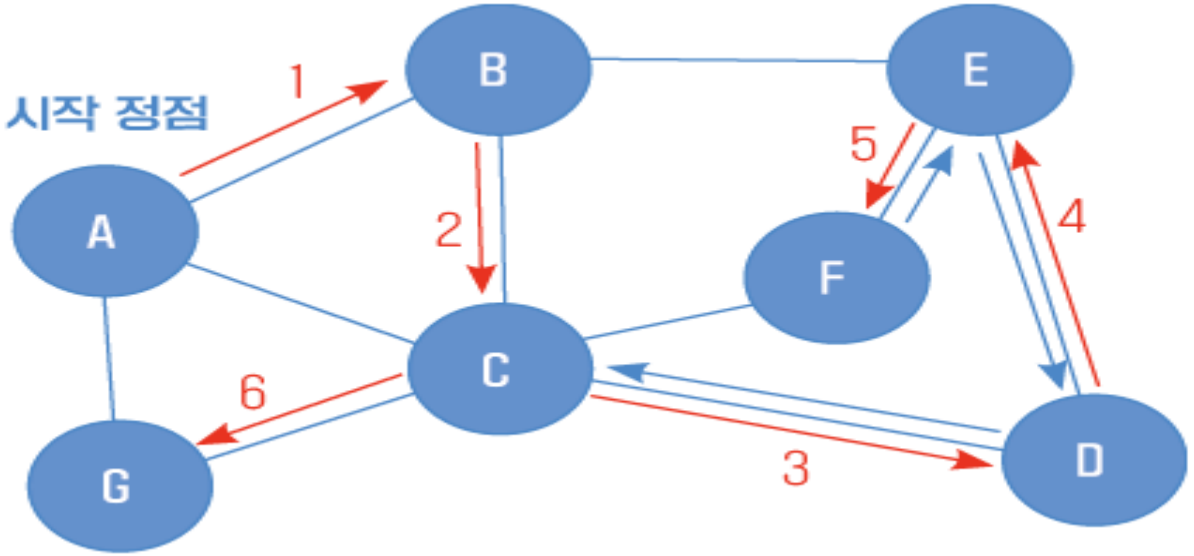




graph_array={A:[B,C,G],B:[A,C,E],C:[A,B,D,F,G],
D:[C,E],E:[B,D,F],F:[C,E],G:[A,C]}

깊이 우선 순회 방법과 컴퓨터 알고리즘 동작

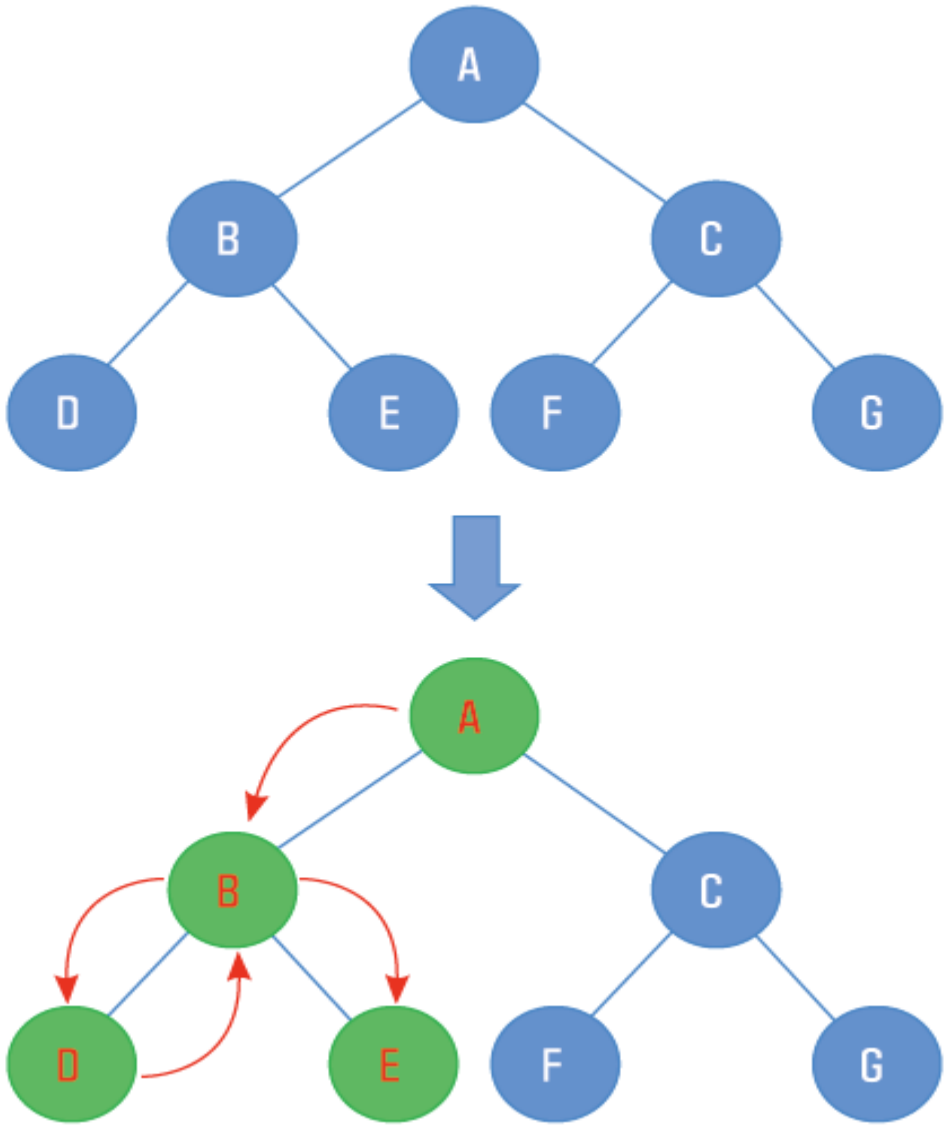
1. 시작 노드를 스택에 삽입하고 방문 처리한다.
2. 스택의 최상위 노드에 방문하지 않은 인접 노드가 있으면 인접 노드를 스택에 삽입하고 방문 처리한다.
3. 스택의 최상위 노드에 방문하지 않은 인접 노드가 없으면 스택에서 삭제한다.
4. 스택이 빌 때까지 수행한다. (스택의 마지막 노드는 시작 노드이다.)



[A] → [A,B] (push) → [A,B,C] (push)
→ [A,B,C,D] (push) → [A,B,C,D,E] (push)
→ [A,B,C,D,E,F] (push) → [A,B,C,D,E] (pop)
→ [A,B,C,D] (pop) → [A,B,C] (pop)
→ [A,B,C,G] (push) → [A,B,C] (pop)
→ [A,B] (pop) → [A] (pop) → [] (pop)

그래프 순회

- 트리구조의 깊이 우선 순회



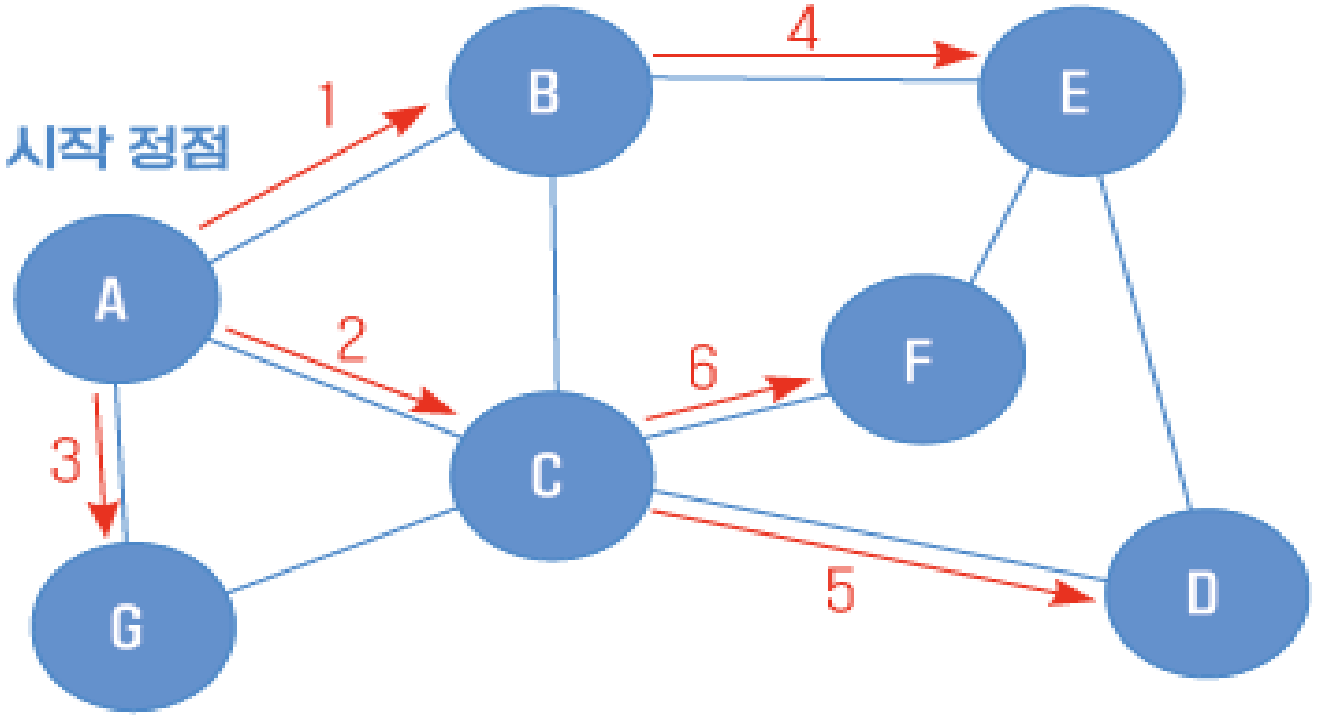
▪ 너비 우선 순회(BFS: Breadth First Search)

➢ **시작 노드를 정하고 인접한 노드를 모두 방문하는 순회 방법**

- 현재 노드의 인접 노드를 모두 방문한다.
- 인접 노드 중 더 이상 방문하지 않은 인접 노드가 존재하지 않으면 방문했던 경로를 되돌아간다.
- 방문하지 않은 노드가 더 이상 존재하지 않으면 종료한다.

너비 우선 순회 방법

- **시작 노드와 가까운 노드부터 방문하고 멀리 떨어진 노드를 나중에 방문하는 순회 방법**
- 시작 노드를 A로 정했다고 가정함. A노드를 가정 먼저 방문한 후 인접한 노드들을 순서대로 모두 방문함. 이때 순서를 정하는 조건은 있을 수도 있고, 없을 수도 있음.
  - 여기에서는 없다고 가정함

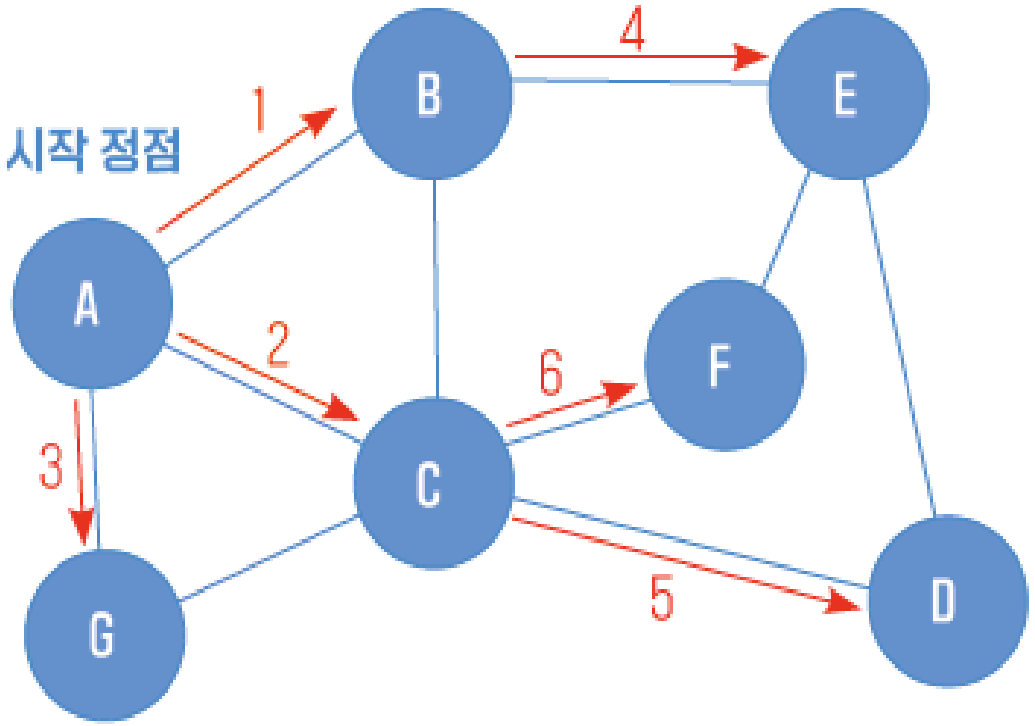


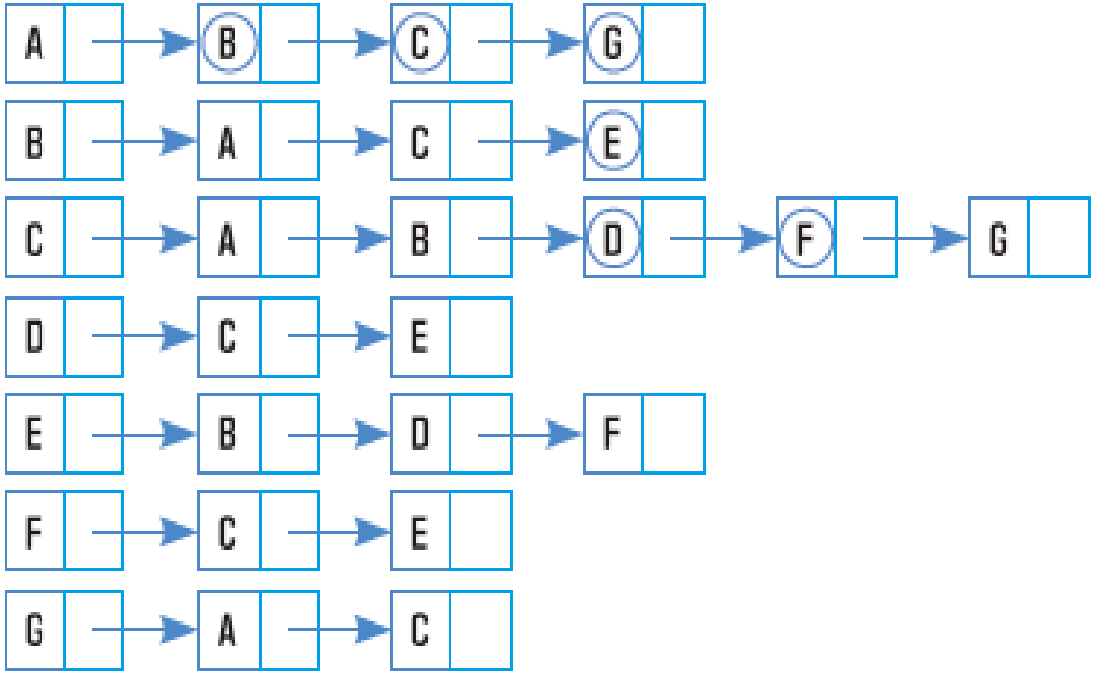
순회 순서

A → B → C → G → B → E → C → D → F → E → D → F

종료 조건(노드 순회 확인)
더 이상 방문할 노드 없음

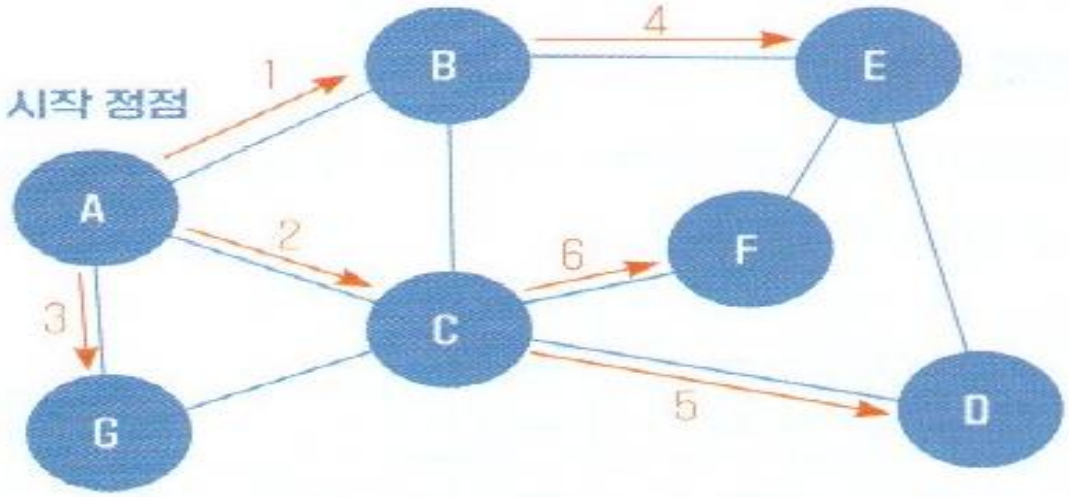
- 너비 우선 순회 방법과 컴퓨터 알고리즘 동작
  - 주어진 그래프를 기본 자료구조인 인접리스트로 표현하여 너비 우선 순회를 수행





graph_array={A:[B,C,G],B:[A,C,E],C:[A,B,D,F,G],
D:[C,E],E:[B,D,F],F:[C,E],G:[A,C]}

- 너비 우선 순회 방법과 컴퓨터 알고리즘 동작
  - 주어진 그래프를 기본 자료구조인 2차원 배열로 표현하여 너비 우선 순회를 수행
    - A노드를 시작으로 A 노드에 포함된 값 중에서 값이 1인 모든 노드를 방문하고, 방문한 순서대로 이동하여(B ->C->G) 다시 값이 1인 노드를 확인함.
    - 위 과정을 반복하여 값이 1인 노드중 방문하지 않은 노드가 없는 것을 확인한 후 순회를 종료함



	A	B	C	D	E	F	G
A	0	1	1	0	0	0	1
B	1	0	1	0	1	0	0
C	1	1	0	1	0	1	1
D	0	0	1	0	1	0	0
E	0	1	0	1	0	1	0
F	0	0	1	0	1	0	0
G	1	0	1	0	0	0	0

graph_array=[[0,1,1,0,0,0,1], [1,0,1,0,1,0,0], [1,1,0,1,0,1,1], [0,0,1,0,1,0,0], [0,1,0,1,0,0,0], [0,0,1,0,1,0,0], [1,0,1,0,0,0,0]]

너비 우선 순회 방법과 컴퓨터 알고리즘 동작

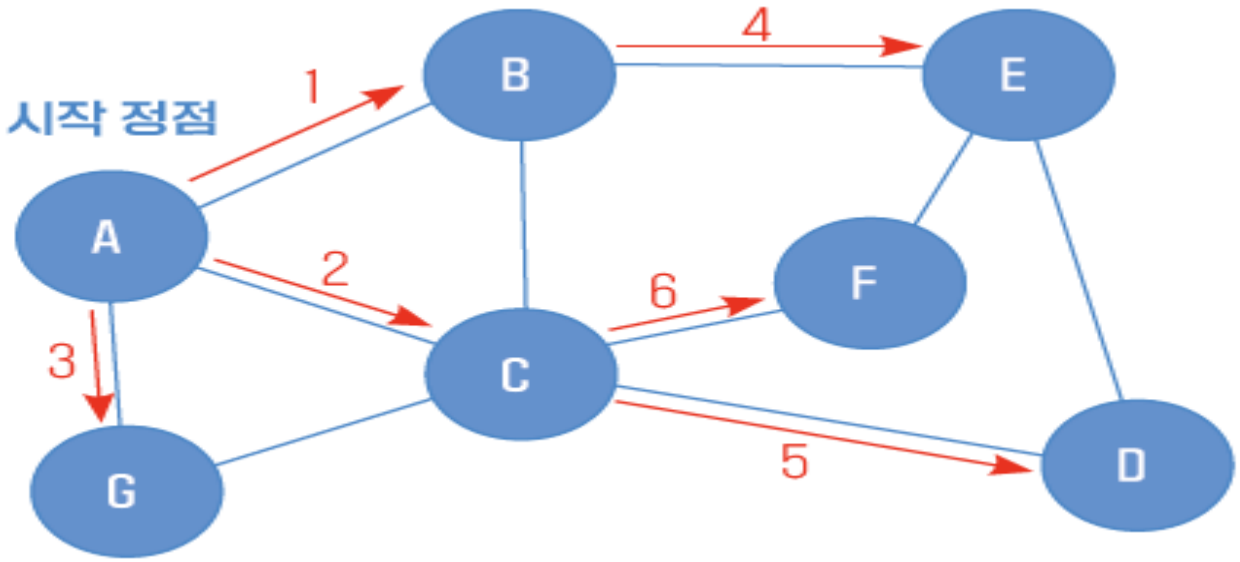
- 큐(Queue) 자료구조를 사용하여 구현하면 쉬움
- 큐는 방문한 노드들을 저장하여 그래프 순회의 종료를 확인하는데 사용함
- 큐를 이용한 너비 우선 순회 방법의 컴퓨터 알고리즘 동작은 아래와 같음

1. 시작 노드를 큐에 삽입하고 방문 처리한다.
2. 큐에서 노드를 꺼내고 해당 노드의 인접 노드 중에서 방문하지 않은 노드를 모두 큐에 삽입하고 방문 처리한다.
3. 큐가 빌 때까지 수행한다. (큐의 마지막 노드는 마지막으로 방문한 노드이다.)

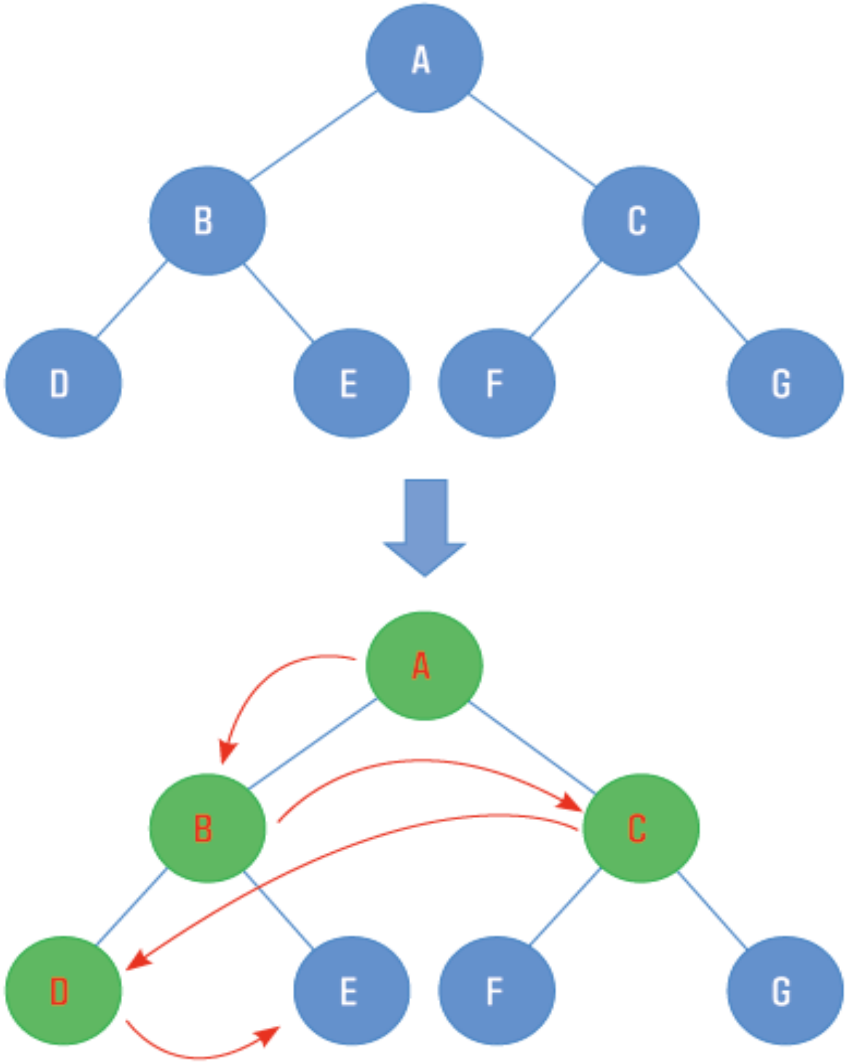
그래프 순회

너비 우선 순회 방법과 컴퓨터 알고리즘 동작

1. 시작 노드를 큐에 삽입하고 방문 처리한다.
2. 큐에서 노드를 꺼내고 해당 노드의 인접 노드 중에서 방문하지 않은 노드를 모두 큐에 삽입하고 방문 처리한다.
3. 큐가 빌 때까지 수행한다. (큐의 마지막 노드는 마지막으로 방문한 노드이다.)



[A] (enqueue)
[B,C,G] (dequeue, enqueue)
[C,G,E] (dequeue, enqueue)
[G,E,D,F] (dequeue, enqueue)
[E,D,F] (dequeue)
[D,F] (dequeue)
[F] (dequeue)

그래프 순회

- 트리 구조의 너비 우선 순회